Homework 4: Direct Preference Optimization for GPT-2; 11-411/611

SGD

Overview

In this homework you will implement a simplified *Direct Preference Optimization* (DPO) trainer for GPT-2. The main goal is to make you more independent at building modern NLP training pipelines: loading preference data, preparing causal language-model inputs, computing sequence log-probabilities, implementing the DPO objective, and writing a training loop from scratch.

You may use `torch` and Hugging Face *model/tokenizer* classes, but you may **not** use any high-level training wrapper such as `transformers.Trainer`, `trl.DPOTrainer`, or any other library that already implements the training objective for you.

This is a **coding-only** assignment. You only need to implement and submit `submission.py`; there is no separate written report.

Learning goals

By the end of this homework, you should be able to:

1. convert preference triples into causal LM training examples;
2. compute token-level and sequence-level log-probabilities correctly;
3. implement the DPO loss from its mathematical definition;
4. train a policy model against a frozen reference model;
5. evaluate preference alignment using a log-probability-based metric.

Background

A preference dataset consists of triples

$$(x, y_w, y_l),$$

where x is a prompt, y_w is the *chosen* response, and y_l is the *rejected* response.

We will train a policy model π_θ relative to a frozen reference model π_{ref}. For a response $y = (y_1, \ldots, y_T)$, define the response log-probability conditioned on prompt x as

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t}).$$

In this homework, you should compute this quantity over the *response tokens only*, not the prompt tokens.

The DPO objective for one preference pair is

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\big(\beta\,\big[(\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)) - (\log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x))\big]\big)\,,$$

where $\sigma(\cdot)$ is the logistic sigmoid and $\beta > 0$ controls how strongly we enforce the preference.

Intuitively, the policy should assign higher relative probability to the chosen response than to the rejected response, but it is anchored by the reference model. A deeper derivation of why this loss arises from first principles — including where the β terms come from, why the partition function cancels, and what the gradient is actually doing — is in Appendix A.

A note on causal LM labels

For a causal LM, the model predicts token $t+1$ from positions up to t. If your input sequence is

$$[\text{prompt tokens}, \text{response tokens}, \text{EOS}],$$

then your `labels` tensor should:

- contain `-100` for prompt positions,
- contain the response token IDs and EOS token ID at the response positions,
- contain `-100` at any padding positions.

This means the loss only applies to the response.

Assignment simplifications

To keep the assignment focused and manageable, we will make the following simplifications:

1. Single-turn prompts only.
2. A standard causal LM interface (GPT-2 family).
3. No PPO, no reward model, and no sampling inside the training loop.
4. Full-model fine-tuning is fine; parameter-efficient methods are not the point of this homework.

Files provided

You are given the following files:

- `submission.py`: starter code with TODOs.
- `train.py`: a runnable training script that calls your implementations.
- `data.py`: helpers for reading JSONL preference data.
- `tests_public/`: local pytest tests you should run before submitting (see Running the public tests).

Running the public tests

The `tests_public/` directory exercises your `submission.py` code (masking, collation, log-probabilities, DPO loss, and related helpers). **Always run commands from the assignment root**—the directory that contains `submission.py`, `pytest.ini`, and `tests_public/` (the same place you run `train.py`).

1. Install Python dependencies, including `pytest`, e.g. `pip install -r requirements.txt` or your course conda/venv setup.
2. Run the full public suite. The quiet form is:

   ```
   pytest -q tests_public
   ```

   This repository also ships `pytest.ini` with `testpaths = tests_public`, so from the assignment root you may equivalently run `pytest -q`.
3. If a test fails, rerun with `-v` (verbose) or without `-q` to see tracebacks and assertion messages, e.g. `pytest tests_public -v`. To run a single file, e.g. `pytest -q tests_public/test_public_dpo_loss.py`.

Passing the public tests is required before you rely on training or the autograder; staff may also run additional private tests.

What you need to implement

Implement the TODOs in `submission.py`. Each function below is a self-contained step in the DPO pipeline; they are meant to be implemented in order, as later functions build on earlier ones.

1. `build_lm_sequence` Concatenate prompt and response token IDs into a single causal LM training example. The returned `labels` must contain `-100` at every prompt position so that the loss is computed only over the response. When the combined sequence exceeds `max_length`, truncate the *prompt* from the left rather than the response, and always ensure the sequence ends with EOS.

2. `tokenize_preference_example` Tokenize one `{"prompt", "chosen", "rejected"}` record. Apply `format_prompt` to the raw prompt string, tokenize each field separately with the provided length budgets, and call `build_lm_sequence` for both the chosen and rejected responses. Return the four lists: `chosen_input_ids`, `chosen_labels`, `rejected_input_ids`, `rejected_labels`.

3. `preference_collate_fn` Collate a list of tokenized examples into a padded batch. A helper stub `_pad_sequences(sequences, *, pad_value)` is declared in `submission.py` for you to implement: given a list of variable-length integer sequences it should return a padded `LongTensor` of shape `[batch, max_len]` and a matching `attention_mask` tensor. Once implemented, call it separately for `input_ids` (pad with `pad_token_id`) and `labels` (pad with `-100`) so that each field receives the correct pad value.

4. `sequence_logps_from_logits` Compute one scalar log-probability per sequence from causal-LM logits and labels. Properly align logits and labels for next-token prediction, ignore masked positions, and either sum or average the per-token log-probabilities over the response positions depending on the `average_log_prob` flag.

5. `dpo_loss` Implement the per-example DPO loss exactly as stated in the Background section. In addition to the loss tensor, return a `stats` dict containing detached tensors: `chosen_rewards`, `rejected_rewards`, `margins`, and `accuracy`.

6. `compute_dpo_batch` Run the policy and reference models on one batch, extract sequence log-probabilities for all four forward passes (policy chosen, policy rejected, reference chosen, reference rejected), and return the mean DPO loss and aggregated metrics. Only the policy model should participate in gradient tracking.

7. `train_step` Perform one gradient update: zero gradients, call `compute_dpo_batch`, backpropagate, optionally clip gradients, and step the optimizer. Return a dict of Python floats with at least `loss`, `preference_accuracy`, and `mean_margin`.

8. `evaluate_preference_accuracy` Iterate over a full dataloader, accumulate metrics across all batches without tracking gradients, and return their averages as Python floats.

Implementation hints

These hints point out non-obvious pitfalls; they do not tell you *how* to implement each function.

- **Causal shift.** A causal LM's logit at position t predicts token $t + 1$. Before you look up log-probabilities using your label values, think carefully about which positions in the logits tensor actually correspond to which labels.
- **Prompt truncation direction.** When the prompt must be shortened to make room for the response, consider *which end* carries the information most relevant to the next token.
- **Attention mask and the EOS token.** GPT-2 reuses the EOS token as its pad token. A mask built by comparing token IDs against `pad_token_id` will silently mark the legitimate EOS at the end of every real sequence as padding. Think about when you have the ground-truth sequence length available.
- **Gradient scope.** The reference model must stay frozen throughout training. One context manager in PyTorch is designed exactly for this; applying it to the right forward pass will prevent any gradient from flowing into the reference model's parameters.
- **Implicit rewards.** The "chosen reward" and "rejected reward" statistics you return from `dpo_loss` are not free-standing scalars you need to compute separately—they fall directly out of quantities you already have at that point. Re-read the DPO loss formula in ratio form (Appendix A, Eq. A.3) for a nudge.
- **Reducing log-probs to a scalar.** `sequence_logps_from_logits` must return one number per sequence. Before calling `dpo_loss`, double-check that your tensor has shape `[batch]` and not `[batch, seq_len]`.

Dataset

For the real experiment, use the course-provided JSONL subset:

```
data/uf_small_train.jsonl
data/uf_small_val.jsonl
data/uf_small_test.jsonl
```

Each row has the form

```
{"prompt": "...", "chosen": "...", "rejected": "..."}
```

This subset is derived from `HuggingFaceH4/ultrafeedback_binarized`. We provide a subset rather than the full source dataset so that:

- you do not spend your time on dataset cleaning,
- everyone trains on the same size budget,
- the default setup fits comfortably in a standard Colab workflow.

Suggested defaults. A good default configuration is:

Model	`gpt2`
β	0.1
Max prompt tokens	96
Max response tokens	64
Max total length	160
Batch size	4
Gradient accumulation	4
Learning rate	1×10^{-5}
Epochs	1 or 2

Why default to one epoch? The provided subset is small, and DPO updates the full policy on every preference pair each epoch. One epoch keeps runtime and memory pressure modest (including Colab) while still showing a clear preference-accuracy trend; training longer can yield slightly better alignment but also risks overfitting this slice of UltraFeedback. You are welcome to train for two epochs if you have the time and want to explore the trade-off.

Training command. From the assignment root (the directory that contains `train.py` and the `data/` folder), run:

```
python train.py \
  --train_path data/uf_small_train.jsonl \
  --val_path data/uf_small_val.jsonl \
  --output_dir outputs/hw4_dpo \
  --model_name gpt2 \
  --beta 0.1 \
  --max_prompt_length 96 \
  --max_response_length 64 \
  --max_length 160 \
  --batch_size 4 \
  --grad_accum_steps 4 \
  --learning_rate 1e-5 \
  --epochs 1 \
  --seed 13
```

This matches the suggested defaults in the table (including gradient clipping at `train.py`'s default of `1.0`). For a second epoch, set `--epochs 2`. The trained policy and tokenizer are written to `outputs/hw4_dpo/policy_model`; metrics are saved as `outputs/hw4_dpo/metrics.json`.

Running the training loop (recommended for exposure). Your grade comes from the autograder on `submission.py`; you are not required to train to full convergence for credit. Even so, we strongly encourage you to run `train.py` at least once on your own machine or Colab: it is the most direct way to see how your DPO implementation behaves in practice—validation metrics, saved `metrics.json`, and the printed greedy completions let you compare the **policy** after updates to the frozen **reference** and get a feel for whether preference alignment is moving in the right direction.

Trying other base models. The table defaults to `gpt2` for speed and modest memory. If you have headroom, try other Hugging Face causal language models (e.g. `distilgpt2`, `gpt2-medium`) by changing `--model_name`. Larger models train more slowly and may need a smaller batch size or shorter max lengths to fit in memory.

Logging and sample generations. `train.py` prints a few greedy-decoded completions after each epoch, comparing the **reference** model (frozen at initialization) to the **policy** after training so far. Prompts are drawn from the validation JSONL with a fixed shuffle controlled by `--seed`.

Weights & Biases (optional). Optional Weights & Biases logging: each epoch records validation metrics and a table `epoch/generations` (prompt, reference, policy). The `wandb` package is listed in `requirements.txt`; omit the `--wandb_project` and `--wandb_run_name` arguments if you do not want W&B.

From the assignment root (the directory that contains `train.py` and `data/`), run the following as a single shell command (it may wrap across lines in this PDF). The first time you use W&B on a laptop, follow any authentication prompt from the client.

```
python train.py --train_path data/uf_small_train.jsonl --val_path data/uf_small_val.jsonl --
    output_dir outputs/hw4_dpo --model_name gpt2 --beta 0.1 --max_prompt_length 96 --
    max_response_length 64 --max_length 160 --batch_size 4 --grad_accum_steps 4 --learning_rate 1e
    -5 --epochs 1 --seed 13 --wandb_project nlp-hw4-dpo --wandb_run_name my-dpo-run
```

Clusters and batch jobs. In the *same* shell before training, set your API key, then run the same command as above:

```
export WANDB_API_KEY="paste-your-key-here"
```

The `wandb` client reads `WANDB_API_KEY` automatically; you do not need to change `train.py`. Rename `nlp-hw4-dpo` or `my-dpo-run` if you prefer different project or run names.

Implementation rules

- You may use `torch`, `transformers` model/tokenizer classes, and ordinary optimizer utilities.
- You may **not** use `transformers.Trainer`, `trl`, `trl.DPOTrainer`, or any package that already implements DPO training for you.
- Your reference model must remain frozen.
- Your code should be vectorized where possible.

- Your public and private tests should pass without editing the test files.

Evaluation

Your homework will be evaluated entirely by the autograder:

1. **Public tests** (local): masking, padding, log-probabilities, and DPO loss.
2. **Private function tests**: additional edge cases and training-step checks.
3. **Private end-to-end test**: a hidden miniature training run compares your implementation against a staff reference on preference-alignment metrics.

Hidden end-to-end metric. On a hidden miniature preference dataset, the autograder will initialize a tiny causal LM, run your training loop for a fixed small number of updates, and compute preference accuracy:

$$\frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\Big[(\log \pi_\theta(y_w^{(i)} \mid x^{(i)}) - \log \pi_\theta(y_l^{(i)} \mid x^{(i)})) > (\log \pi_{\text{ref}}(y_w^{(i)} \mid x^{(i)}) - \log \pi_{\text{ref}}(y_l^{(i)} \mid x^{(i)}))\Big].$$

Your score will be based on how closely your result matches the staff implementation.

Submission checklist

Submit:

1. `submission.py`

This is a coding-only assignment. Submit `submission.py` to Gradescope; all other grading will be handled automatically by the autograder.

Before submitting, make sure you can run (see Running the public tests):

```
pytest -q tests_public
python train.py --help
```

and that the training command in the **Dataset** section completes without errors.

Common pitfalls

- **Forgetting the causal shift.** Sequence log-probabilities must be computed from shifted logits and shifted labels.
- **Scoring the prompt.** Only response tokens should contribute to the LM objective.
- **Incorrect padding masks.** If you use GPT-2, it is common to set `pad_token = eos_token`; your attention mask should therefore be built from sequence lengths, not from token equality checks.
- **Updating the reference model.** It should stay frozen.
- **Using a high-level trainer.** The point of this assignment is to write the pipeline yourself.

Citations

- Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*.
- Cui et al., *UltraFeedback: Boosting Language Models with High-quality Feedback*.

Appendix: Intuition for DPO

A.1 Why skip the reward model?

The standard RLHF pipeline has two stages: (1) fit a scalar reward model $r(x, y)$ from human preference labels, then (2) run reinforcement learning to find a policy with high reward while staying close to a reference model. DPO collapses both stages into a single supervised loss. To see why this is possible, start with the KL-penalized reward-maximization objective:

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] \;-\; \beta\, D_{\mathrm{KL}}\big[\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\big]. \tag{A.1}$$

The first term wants high reward; the second term keeps the policy close to the reference. This objective has a closed-form solution:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x)\, \exp\!\left(\frac{r(x,y)}{\beta}\right), \quad Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \exp\!\left(\frac{r(x,y)}{\beta}\right). \tag{A.2}$$

Equation (A.2) says the optimal policy is just the reference model *reweighted* by the exponentiated reward. Responses with higher reward get a larger multiplicative boost; $Z(x)$ is a prompt-level normalization constant that ensures the probabilities sum to 1.

A.2 Reward as a log-probability ratio

Rearranging (A.2) to isolate r gives:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x). \tag{A.3}$$

This is the key insight. *Up to a prompt-specific constant $\beta \log Z(x)$, the reward of a response equals how much more likely it is under the optimal policy than under the reference model.* We do not need an explicit reward model; the policy's own probabilities encode the reward implicitly.

A.3 The partition function cancels

The Bradley–Terry model assigns probability to the preference $y_w \succ y_l$ as

$$p(y_w \succ y_l \mid x) = \sigma\big(r(x, y_w) - r(x, y_l)\big). \tag{A.4}$$

Substituting (A.3) into (A.4), the term $\beta \log Z(x)$ appears in both $r(x, y_w)$ and $r(x, y_l)$ and therefore *cancels*:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}. \tag{A.5}$$

We never need to compute or estimate $Z(x)$. Maximum-likelihood training on the preference labels therefore gives the DPO objective:

$$\mathcal{L}_{\mathrm{DPO}} = -\log \sigma\!\left(\beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right]\right), \tag{A.6}$$

which is exactly the DPO loss in the Background section written in ratio form.

A.4 What the gradient is doing

Define the *implicit reward difference* for a pair as

$$\hat{r}_\theta(x, y_w, y_l) = \beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right].$$

The gradient of $\mathcal{L}_{\text{DPO}}$ with respect to θ is

$$-\beta\,(1 - \sigma(\hat{r}_\theta))\,[\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)]. \tag{A.7}$$

This has two parts:

- The direction $[\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)]$ always pushes the winner up and the loser down.
- The scalar weight $(1 - \sigma(\hat{r}_\theta))$ is large when the model is "wrong" (i.e., $\hat{r}_\theta$ is small or negative, meaning the model currently ranks the rejected response too highly) and small when the model is already correct. DPO thus concentrates its updates on the pairs it currently gets most wrong.

A.5 Plain-language summary

Intuition for DPO. Direct Preference Optimization avoids the usual two-stage RLHF pipeline of first learning a reward model and then optimizing it with reinforcement learning. Instead, it exploits the fact that under a KL-constrained objective the optimal policy can be written as the reference policy reweighted by reward (Eq. A.2). This means that, up to a prompt-specific constant, the reward of a response is equivalent to how much more likely that response is under the learned policy than under the reference model (Eq. A.3). Since the Bradley–Terry preference model depends only on *reward differences* between two responses, the prompt-specific constant cancels (Eq. A.5), and DPO can train the policy directly on preference pairs without fitting a separate reward model or running RL. The update is strongest when the model currently assigns too much implicit reward to the rejected response (Eq. A.7), making DPO a simple and stable way to fine-tune a model from preference data.

A one-line summary: **DPO = pairwise logistic regression over responses, where the "score" is the model's log-probability gain over the reference model.**

Gradescope submission

Submit your work on Gradescope. For this coding-only assignment, the only file you need to upload is `submission.py`; everything else will be handled for you. The autograder will run the public and private tests and assign your marks automatically.